

Module 11 Lab Session 1: Password Hashing Mechanics & ASP.NET Core Identity Setup

Session 1 of 3

Exercises Exercise 1 & Exercise 2

Integrity Lab tier **Tier 1** Salting & hashing mechanics, IdentityUser extension, IdentityDbContext, password policy, logout config

Story thread

Before configuring ASP.NET Core Identity, you will use BCrypt.Net-Next to observe how salts prevent rainbow table lookups and slow down brute-force attacks. Next, you will integrate **ASP.NET Core Identity** into the TMS API to manage TmsUser accounts, enforce enterprise password policies, handle brute-force lockouts, and run database migrations.

Exercise 1: The Mechanics of Password Hashing

Context: Plain-text passwords and unsalted hashes (like raw MD5 or SHA256) are major security vulnerabilities. BCrypt uses a salt and work factor to slow down brute-force attacks.

Step 1: Install BCrypt Package

Open a terminal in your Web API project directory (Tms.Api):

```
Dotnet add TmsApi.Infrastructure/TmsApi.Infrastructure.csproj package BCrypt.Net-Next
```

Step 2: Create Cryptography Service

Create TmsApi.Infrastructure/Services/CryptoDemoService.cs:

```
namespace TmsApi.Infrastructure.Services;

public class CryptoDemoService
{
    public string HashUserPassword(string plainText)
    {
        // BCrypt automatically generates a unique salt and prepends it
        // to the hash.
        // workFactor: 12 means 2^12 key expansion iterations.
    }
}
```

```

        return BCrypt.Net.BCrypt.HashPassword(plainText, workFactor: 12);
    }

    public bool VerifyUserPassword(string plainText, string hashedDbPassword)
    {
        return BCrypt.Net.BCrypt.Verify(plainText, hashedDbPassword);
    }
}

```

Step 3: Test Salt Uniqueness

Inspect in an endpoint or test method:

```

var service = new CryptoDemoService();
string hash1 = service.HashUserPassword("Password123!");
string hash2 = service.HashUserPassword("Password123!");

// hash1 and hash2 are completely different strings because of unique random salts!
Console.WriteLine($"Hash 1: {hash1}");
Console.WriteLine($"Hash 2: {hash2}");

// Both verify to true against the same plain text:
bool match1 = service.VerifyUserPassword("Password123!", hash1); // true
bool match2 = service.VerifyUserPassword("Password123!", hash2); // true

```

Decision Rule: Never write custom hashing algorithms in production. In Exercise 2 onwards, you will use UserManager provided by ASP.NET Core Identity.

Exercise 2: ASP.NET Core Identity Setup & AuthController

Step 1: Extend IdentityUser

Create TmsApi.Domain/Entities/TmsUser.cs in your API project:

```

using Microsoft.AspNetCore.Identity;

namespace TmsApi.Domain.Entities;

public class TmsUser : IdentityUser
{
    public string FirstName { get; set; } = string.Empty;
    public string LastName { get; set; } = string.Empty;
    public string? Department { get; set; }
}

```

Step 2: Update DbContext

In `TmsApi.Infrastructure/Persistence/TmsDbContext.cs`, update your context to inherit from `IdentityDbContext<TmsUser>`:

```
namespace TTmsApi.Infrastructure.Persistence;

public class TmsDbContext : IdentityDbContext<TmsUser>
{
    public TmsDbContext(DbContextOptions<TmsDbContext> options) :
    base(options) { }

}
```

Step 3: Configure Identity Services in Program.cs

Open `Program.cs` and configure Identity Core with enterprise password and lockout policies:

```
builder.Services.AddIdentityCore<TmsUser>(options =>
{
    // Enterprise Password Policy
    options.Password.RequiredLength = 12;
    options.Password.RequireUppercase = true;
    options.Password.RequireDigit = true;
    options.Password.RequireNonAlphanumeric = true;

    // Brute-Force Lockout Protection
    options.Lockout.MaxFailedAccessAttempts = 5;
    options.Lockout.DefaultLockoutTimeSpan = TimeSpan.FromMinutes(15);
    options.Lockout.AllowedForNewUsers = true;
}))
.AddRoles<IdentityRole>()
.AddEntityFrameworkStores<TmsDbContext>();
```

Step 4: Replace the Demo AuthController with Identity

Replace the M10 demo `AuthController` with the Identity-backed version below. The M10 version used hardcoded credentials for transport testing this version uses `UserManager` for real accounts.

Update `TmsApi.Api/Controllers/AuthController.cs`:

```
namespace TmsApi.Api.Controllers;

[ApiController]
[Route("api/[controller]")]
public class AuthController : ControllerBase
```

```

{
    private readonly UserManager<TmsUser> _userManager;
    private readonly RoleManager<IdentityRole> _roleManager;

    public AuthController(
        UserManager<TmsUser> userManager,
        RoleManager<IdentityRole> roleManager)
    {
        _userManager = userManager;
        _roleManager = roleManager;
    }

    public record RegisterRequest(
        string Email,
        string Password,
        string FirstName,
        string LastName,
        string Role);

    [HttpPost("register")]
    public async Task<IActionResult> Register([FromBody]
RegisterRequest request)
    {
        var existingUser = await
_userManager.FindByEmailAsync(request.Email);
        if (existingUser != null)
        {
            // Prevent account enumeration by returning a generic
response
            return Ok(new { message = "Registration request
received." });
        }

        var user = new TmsUser
        {
            UserName = request.Email,
            Email = request.Email,
            FirstName = request.FirstName,
            LastName = request.LastName
        };

        var result = await _userManager.CreateAsync(user,
request.Password);
        if (!result.Succeeded)
        {
            var errors = result.Errors.Select(e => e.Description);
            return BadRequest(new { errors });
        }
    }
}

```

```

        // Ensure requested role exists
        if (!await _roleManager.RoleExistsAsync(request.Role))
        {
            await _roleManager.CreateAsync(new
IdentityRole(request.Role));
        }

        await _userManager.AddToRoleAsync(user, request.Role);
        return Ok(new { message = "Registration successful." });
    }

    public record LoginRequest(string Email, string Password);

    [HttpPost("login")]
    public async Task<IActionResult> Login([FromBody] LoginRequest
request)
    {
        var user = await _userManager.FindByEmailAsync(request.Email);
        if (user == null)
        {
            return Unauthorized(new { detail = "Invalid
credentials." });
        }

        if (await _userManager.IsLockedOutAsync(user))
        {
            return StatusCode(423, new { detail = "Account locked due
to multiple failed login attempts. Try again in 15 minutes." });
        }

        var validPassword = await _userManager.CheckPasswordAsync(user,
request.Password);
        if (!validPassword)
        {
            await _userManager.AccessFailedAsync(user);
            return Unauthorized(new { detail = "Invalid
credentials." });
        }

        // Reset failed attempt counter on successful login
        await _userManager.ResetAccessFailedCountAsync(user);

        return Ok(new
        {
            userId = user.Id,
            email = user.Email,
            firstName = user.FirstName,

```

```

        lastName = user.LastName
    });
}
}

```

Step 5: Run Database Migration

Run migrations from your solution root targeting the Infrastructure and Api projects:

```
dotnet ef migrations add AddIdentitySupport --project TmsApi.Infrastructure/TmsApi.Infrastructure.csproj --startup-project TmsApi.Api/TmsApi.Api.csproj
```

```
dotnet ef database update --project TmsApi.Infrastructure/TmsApi.Infrastructure.csproj --startup-project TmsApi.Api/TmsApi.Api.csproj
```

Expected result: The migration creates seven standard ASP.NET... tables (AspNetUsers, AspNetRoles, AspNetUserClaims, etc.) in your database.

Verification Checkpoint (Session 1)

1. Scalar / HTTP Verification:

- Open OpenAPI / Scalar at <https://localhost:5001/scalar/v1>.
- Send POST /api/auth/register with body:

```

{
  "email": "leul.instructor@cotbe.edu.et",
  "password": "SecurePass123!",
  "firstName": "Leul",
  "lastName": "Gebre",
  "role": "Instructor"
}

```

- Confirm response 200 OK.

2. Password Validation Check:

- Attempt registration with password "short". Confirm 400 Bad Request listing password policy errors.

3. Lockout Check:

- Attempt POST /api/auth/login with wrong password 5 times. On 6th attempt, confirm status 423 Locked returned.